K80, another Z80 SBC with RC2014 expansion on 100mmX100mm board

644 views Subscribe

Bill Shen Aug 28, 2019, 2:43:09 PM
to retro-comp

I thought I've introduced my K80 SBC here or at RC2014-Z80 forum, but I can't find it, so I'm reintroducing it as a new topic.

https://www.retrobrewcomputers.org/doku.php?id=builderpages:plasmo:k80

K80 is a basic Z80/ROM/RAM computer but with KIO for I/O. It is unremarkable except for the KIO which is an integrated PIO/SIO/CTC peripheral designed to run to 16MHz. I designed the memory interface to get as much access time as I can, and I separate the CPU clock from I/O clock so I can change the CPU clock at will.

The reason I revisited K80 is because a couple persons had the pc board built and are currently assembling their own K80. In the process I found a bug in the CPLD equations that caused problems with certain brand of compact flash disks. I fixed the bug and ran up the CPU clock to check the design margin and realized it can run reliably to 33MHz at room temperature! At 34.5Mhz, K80 will boot, but the memory diagnostic will fail. At 33MHz, it booted up CP/M and ran ZEXALL.COM in 24 minutes and passed 100%. Current consumption is 170mA@5V

I ran the assembly mandelbrot benchmark, AKA, shark jumping, (https://groups.google.com/forum/#!topic/rc2014-z80/9B1NfijuUBM). It ran in 1 minute and 26 seconds, which is odd, because the 36MHz Z180 ran 1 minute 40 seconds. We may need to revisit the assembly mandelbrot benchmark again.
Bill



DSC_46520811.jpg

Phillip Stevens Aug 28, 2019, 3:42:57 PM
to retro-comp

Bill Shen wrote:
> I thought I've introduced my K80 SBC here or at RC2014-Z80 forum, but I can't find it, so I'm reintroducing it as a new topic.
>
> https://www.retrobrewcomputers.org/doku.php?id=builderpages:plasmo:k80
>
> I ran the assembly mandelbrot benchmark, AKA, shark jumping, (https://groups.google.com/forum/#!topic/rc2014-z80/9B1NfijuUBM). It ran in 1 minute and 26 seconds, which is odd, because the 36MHz Z180 ran 1 minute 40 seconds. We may need to revisit the assembly mandelbrot benchmark again.

Always happy to go shark jumping. I had a mental issue with setting the Z180 to the correct clock multiplier, and the benchmarks at the link are not correct. I was actually testing the Z180 at 18.432MHz.

I've attached my current numbers below, with what I think is the minimum time possible (would like your opinion on whether my calculation is correct).

```
; To calculate the theoretical minimum time at 115200 baud.
; Normally 10 colour codes, and 1 character per point.
; A line is (3 x 80) x 11 + CR + LF = 2642 characters
; There are 10 x 60 / 4 lines = 150 lines
; Therefore 396,300 characters need to be transmitted.
; Serial rate is 115200 baud or 14,400 8 bit characters per second
;
; Therefore the theoretical minimum time is 27.52 seconds.
;
; Results          FCPU         Original    Optimised   z180 mlt     APU
; RC2014 CP/M    7.432MHz         4'51"       4'10"
; YAZ180 CP/M   18.432MHz         1'40"       1'24"       1'00"
; YAZ180 yabios 18.432Mhz                     1'14"         54"
; YAZ180 CP/M   36.864MHz                       58"         46"
; YAZ180 yabios 36.864Mhz                       56"         45"
```

I've been playing with floating point code recently. I must rewrite mandelbrot in IEEE754 floating point, and test it for interest.

Cheers, Phillip

Bill Shen Aug 28, 2019, 7:17:13 PM
to retro-comp

Do you have a value for original benchmark with 36.8Mhz clock?

I agree with your calculation, but since I don't have interrupt service in current version of K80 CPM, I'll see all that time whereas your interrupt driver will probably hide half of the time. I'm expecting 10-15 seconds better time from YAZ180.
Bill

Mark T Aug 28, 2019, 7:57:16 PM
to retro-comp

If you include /M1 + /MREQ in the chip enable decoding for memory would that improve the margin on access time?

/M1 is a half clock cycle earlier than /MREQ on fetch cycles which is worst case for memory access. Need to make sure /OE is qualified with /RD to avoid contention during interupt acknowledge.

Mark

Bill Shen Aug 28, 2019, 9:48:47 PM
to retro-comp

I'm leaning toward trading power for speed, I.e., ground the chip selects of ROM and RAM and put logics into output enables and write enable. The access time from output enable is significantly faster than that of chip select. The downside is greater power consumption.
Bill

Mark T Aug 28, 2019, 10:20:43 PM
to retro-comp

I did something similar while trying to figure out why my single board cpm would not run past 12MHz. Used NOT /IORQ to CS, then /M1 AND /RD for /OE. This might need further analysis for interrupt acknowledge as it would rely on the Tdis of the memory being faster that Ten of any interupt vector.

I still need to build up a board without these wire mods to see if it has any significant impact on max clock speed and I've only tested to 20MHz.

Mark



Phillip Stevens Aug 29, 2019, 2:06:36 AM
to retro-comp

Bill Shen wrote:
> Do you have a value for original benchmark with 36.8Mhz clock?

I'll have to see whether the original gist (at the root of the original thread from JB) is still around, and it is.
So we can replicate the original numbers.

My optimisation was unrolling the multiply. I now have a faster table multiply to try, when I'm in the mood to play with sharks.
Perhaps over the weekend, I can play with this, and a floating point version too.

> I agree with your calculation, but since I don't have interrupt service in current version of K80 CPM, I'll see all that time whereas your interrupt driver will probably hide half of the time. I'm expecting 10-15 seconds better time from YAZ180.

Cheers, Phillip

Bill Shen Aug 29, 2019, 2:10:57 PM
to retro-comp

Attached is the source code that I assembled using zmac. Xmodem the .hex output to my K80 and use LOAD command to convert it to brot.com. With 7.37MHz CPU oscillator and my version of CPM (no interrupt), it took 4:44 to run. 1:26 with 33MHz CPU clock.
Bill

mandelbrot.z mandelbrot.hex



Phillip Stevens Aug 29, 2019, 3:10:59 PM
to retro...@googlegroups.com

> Bill Shen wrote:
> > Attached is the source code that I assembled using zmac.
>
> Good to have a standardised version to work from from now on.
>
> > With 7.37MHz CPU oscillator and my version of CPM (no interrupt), it took 4:44 to run.
>
> On my RC2014 using the SIO it took 4'58" to run. Slightly longer. I think we discussed elsewhere that using interrupt driver code is only useful up to a certain point.
>
> > 1:26 with 33MHz CPU clock.
>
> On the YAZ180 at 36.864MHz it takes 1'06". Slightly shorter, quite proportional to the clock rate and perhaps some slightly faster instruction timings, which is expected.
>
> > > My only optimisation was unrolling the multiply.
>
> With the optimisation (old code) it took 58", so that's about 10% improvement.
> Using the z180 multiply instruction took it down to 46".
>
> > > I now have a faster table multiply to try, when I'm in the mood to play with sharks. Perhaps over the weekend, I can play with this, and a floating point version too.
>
> I'll have a further play over the weekend to see what can be done with the multiply routines. I'm trying not to optimise by (for example) only sending a colour code when it changes, as this would completely change the program timings.
>
> Cheers, Phillip



rwd...@gmail.com Aug 29, 2019, 4:49:44 PM
to retro-comp

for comparison mandelbrot.hex ran in 1:41 on my 50mhz multicomp cyclone II (though I saw nothing between start and end messages with xterm terminal type)

Richard



Bill Shen Aug 29, 2019, 7:31:03 PM
to retro-comp

Why is the 50Mhz Z80 running slower than 33MHz? Are there wait states for the memory access?
Bill



Phillip Stevens Aug 31, 2019, 3:26:20 PM
to retro-comp

> Phillip Stevens wrote:
> > Bill Shen wrote:
> > > Attached is the source code that I assembled using zmac.
> >
> > Good to have a standardised version to work from from now on.
> >
> > > With 7.37MHz CPU oscillator and my version of CPM (no interrupt), it took 4:44 to run.
> >
> > On my RC2014 using the SIO it took 4'58" to run. Slightly longer. I think we discussed elsewhere that using interrupt driver code is only useful up to a certain point.
> >
> > I'll have a further play over the weekend to see what can be done with the multiply routines. I'm not trying to optimise by (for example) only sending a colour code when it changes, as this would completely change the program timings.

I've had some time this evening to strip the fast `mulu_de` table look-up `16_8x8` routine out of the IEEE floating point library I've been writing, and match it up to the Spectrum Next `z80n_mulu_32_16x16` multiply. Doing this takes more than a minute off the benchmark for me from 4'58" down to 3'48", whilst still producing all the same 396,300 characters.

Interestingly, it has almost no effect on the z180 time.

```
; Results          FCPU         Original    Optimised   z180 mlt
; RC2014 CP/M    7.432MHz         4'58"       3'48"
; YAZ180 CP/M   36.864MHz         1'06"         58"         46"
; YAZ180 yabios 36.864Mhz                       56"         45"
```

I've attached the code and HEX file, in case anyone wants to try it.

Cheers, Phillip

mandel.asm

mandel.hex

rwd...@gmail.com Aug 31, 2019, 6:07:00 PM
to retro-comp

Using new math code:
Went from 1:50 to 1:26 On my Multicomp 50mhz Cyclone IIc on cpm3
1:41 to 1:26 cpm22

Richard

Bill Shen Aug 31, 2019, 7:58:01 PM
to retro...@googlegroups.com

With the optimized mandel.hex, the execution time is 1 minute 13 seconds on Z80 with 33MHz CPU clock

Bill

etched...@gmail.com Sep 2, 2019, 12:28:04 AM
to retro-comp

On Wednesday, 28 August 2019 13:43:09 UTC+1, Bill Shen wrote:
> I thought I've introduced my K80 SBC here or at RC2014-Z80 forum, but I can't find it, so I'm reintroducing it as a new topic.
>
> https://www.retrobrewcomputers.org/doku.php?id=builderpages:plasmo:k80
>
> K80 is a basic Z80/ROM/RAM computer but with KIO for I/O. It is unremarkable except for the KIO which is an integrated PIO/SIO/CTC peripheral designed to run to 16MHz. I designed the memory interface to get as much access time as I can, and I separate the CPU clock from I/O clock so I can change the CPU clock at will.

Interesting - but confusing. One part says 128K RAM, the schematic says 512K RAM and 16 x 16 mentioned elsewhere is 256K RAM 8)

Is the memory and rtc I/O documented anywhere - if so I can add it to the emulator for completeness.

Alan

Bill Shen Sep 3, 2019, 4:27:24 AM
to retro-comp

It is confusing, mainly because I want to have 32 16K RAM banks, but I ran out of I/O pin on the CPLD, so this design only has 16 16K banks. I have so many irons in the fire, I have not look at the CPLD design so far, but I'm fairly confident I can find the extra I/O pin and internal logic to make it 32 16K banks.
Bill

Bill Shen Nov 18, 2022, 2:51:03 PM
to retro-comp

Well, it took me 3 years and a pandemic to come back to rework K80! I want a RC2014 motherboard with traditional Z80, RAM flash, I/O, CF interface, and RC2014 expansion bus, all in through-hole components. Software-wise I want to install a standard ROMWBW. The existing K80 is close to that goal, so I reworked a rev0 K80 board to check out the idea. The central idea is emulating the bank manager of 512K ROM/RAM board in CPLD. I really want the full feature with 32 16-K banks of RAM and ROM because I want to explore Ladislau RTM/Z80. The full-feature bank manager need more I/O so I have to cannibalize the 3 lines for DS1302 RTC; I will still have RTC function by bit-banging the PIO.

There are quite a number of bodge but it is all working. ROMWBW is the standard RCZ80 with KIO. KIO has its own independent I/O clock so Z80's clock can be changed at will. KIO is rated to 12.5MHz but I know it will run up to 29.5MHz. In the picture the clock is 22MHz and the board is working well. The 8-bit direct-connected CF interface is quite robust. I've tried a number of previously "difficult" CF disks including SD-to-CF adapter and they all worked.

This will be my RC2014 workhorse to explore a number of recent RC2014 projects.
Bill




DSC_70871117.jpg


DSC_70861117.jpg

Wayne Warthen Nov 18, 2022, 8:00:23 PM
to retro-comp

Very nice Bill. A good combination of parts and excellent performance out of a Z80. Of course, I always enjoy seeing another platform running RomWBW! 😀

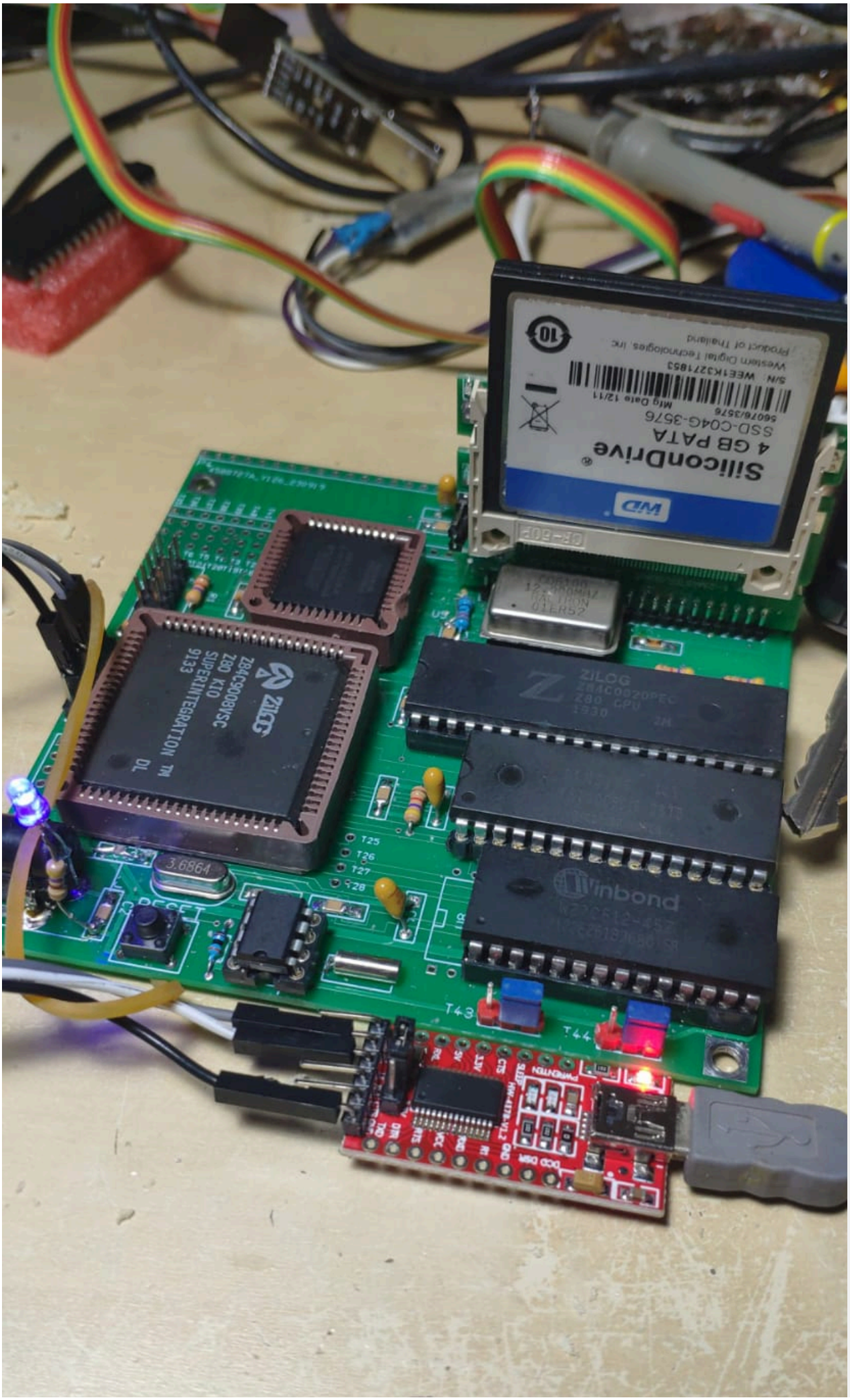
-Wayne



Nicolae Muntean

to retro-comp

Nov 15, 2023, 11:40:32 PM


SiliconDrive
4 GB PATA
SSD-C04G-3576
ZILOG
Z84C0020PEC
Z80 CPU
1930
Winbond
ZILOG
Z84C9008VSC
Z80 KIO
SUPERINTEGRATION
9133
3.6864
RESET





Nicolae Muntean Nov 15, 2023, 11:43:11 PM

to retro-comp

HELLO SIR
I DID THE K80 ACCORDING TO YOUR DOCUMENTATION AND I REACHED THE PHASE WHERE I DON'T UNDERSTAND WHY THE TEST DOESN'T WORK AND MORE.
I SAW A PICTURE WITH UPDATE AND MORE WIRES ON THE BACK OF THE REV 0 BOARD
COULD I THROUGH YOUR GOOD WILL TO RECEIVE THESE MODIFICATIONS TO FINISH THE RESPECTIVE PROJECT?
THANK YOU

Bill Shen Nov 16, 2023, 1:01:57 AM

to retro-comp

This is pictures of my KIO with modifications for it to work. Under the KIO, the two SMT resistors are 1K; the three 1/10W resistors are 4.7K. The resistor at top of picture is 100ohm but not necessary unless you are are using the RC2014 expansion board. The battery holder is for DS1302 RTC, you can ignore it for now. Your oscillator is 12MHz which should work just fine.

I will build another board tonight to verify these changes and update the KIO homepage.
Bill


DSC_74731115.jpg


DSC_74721115.jpg

Bill Shen Nov 16, 2023, 3:15:10 AM

to retro-comp

I build up another K80, make the engineering changes as shown in the picture. I used the CPLD and monitor design files from K80 homepage. It all worked OK. It will boot into monitor without DS1302 and without CF disk. 12MHz clock works just fine. The component side picture shows the minimum components needed to boot into monitor. CF adapter and RTC can be added later.

I will add the engineering change to K80 homepage.
Bill


DSC_74771115.jpg


DSC_74761115.jpg

Bill Shen Mar 25, 2024, 6:07:39 PM

to retro-comp

The various bodges on rev0 K80 is fixed in rev1. The design is also changed to to run RomWBW with 512K RAM/ROM and change in CPLD. The change is significant enough to rename rev1 design to K80W.

One benefit of making K80W capable of RomWBW is the supports for RTC, SD, video/keyboard, and audio functions. The RTC implemented in K80W is the less-known MF/PIC version, but it is quite easy to modify existing RomWBW routines to work with K80W memory map. I'm currently working with Wayne Warthen to incorporate the MF/PIC for K80W into the standard RomWBW configuration process.

I'm also experimenting with SD for K80W. The PIO function is integrated in KIO and RomWBW's SD routine has an option for PIO, so adding SD to K80W is a matter of mapping KIO's PB0 to MOSI, PB7 to MISO, PB3 to CS and PB4 to CLK--no glue logic involved. The memory map for original PIO-based SD routine needs to change to KIO, but that's a small change and K80W now has a SD interface. It is really easy and I should've done this long time ago.

Where I'm going with this is circling back to Eazy80 to add SD function so I can have a Z80SBC with CP/M capability without glue logic
Bill

RomWBW HBIOS v3.4.0-rc.4, 2024-03-25

RCBus [RCZ80_k80w] Z80 @ 20.944MHz
0 MEM W/S, 1 I/O W/S, INT MODE 2, Z2 MMU
512KB ROM, 512KB RAM
ROM VERIFY: 00 00 00 00 PASS

KIO: IO=0x80 ENABLED
CTC: IO=0x84 TIMER MODE=TIM16
SIO0: IO=0x89 SIO MODE=115200,8,N,1
SIO1: IO=0x8B SIO MODE=115200,8,N,1
DSRTC: MODE=MFPIC IO=0xC0 Thu 20FF-1F-3F BF:7F:45 CHARGE=OFF
MD: UNITS=2 ROMDISK=384KB RAMDISK=352KB
FD: MODE=RCWDC IO=0x50 NOT PRESENT
IDE: IO=0x10 MODE=RC
IDE0: NO MEDIA
IDE1: NO MEDIA
PPIDE: IO=0x20 PPI NOT PRESENT
SD: MODE=PIO IO=0x82 DEVICES=1
SD0: SDSC NAME=00000 BLOCKS=0x003BB000 SIZE=1910MB
CH0: IO=0x3E NOT PRESENT
CH1: IO=0x3C NOT PRESENT
FP: IO=0x00 NOT PRESENT

Unit	*Device*	*Type*	*Capacity/Mode*
Char 0	*SIO0:*	*RS-232*	*115200,8,N,1*
Char 1	*SIO1:*	*RS-232*	*115200,8,N,1*
Disk 0	*MD0:*	*RAM Disk*	*352KB,LBA*
Disk 1	*MD1:*	*ROM Disk*	*384KB,LBA*
Disk 2	*IDE0:*	*Hard Disk*	*--*
Disk 3	*IDE1:*	*Hard Disk*	*--*
Disk 4	*SD0:*	*SD Card*	*1910MB,LBA*


K80W with SD.jpg


K80W_rev1_ann…



Bill Shen Dec 27, 2024, 5:10:03 PM
to retro-comp

I've revised K80W to add a SD card interface, fixed RTC noise issue, and added an I2C interface. The CPLD JTAG programming header was relocated so the oscillator can be moved to a better location which, in turn, moved CF interface to avoid possible interference with RC2014 expansion card and created a space for I2C interface. I've also relocated few signal traces that were routed directly under DS1302 RTC to reduce noise interference. However, because there were significant manual routings in this revision that inadvertently caused a missing ground trace from the power jack to rest of the circuit. Thus an engineering change is required to patch in the ground line. The missing ground trace can be easily added, but I'm checking out the rest of the design before releasing the updated Gerber artworks.

The RomWBW sign on message shows it has detected RTC and correctly calculated the CPU oscillator frequency; detected both CF disk as well as SD card.
Bill


DSC_77771227.jpg


K80W_rev2_Ro…


K80_rev2_annot…



Paul dB Dec 27, 2024, 11:46:09 PM
to retro-comp

Looking good Bill!